AuthGraph: Authentication-Aware Graph Neural Networks for Detecting Unauthorized Credential Data Flows in Browser Extensions

Authors: AuthGraph Research Group

Target Venue: IEEE Symposium on Security and Privacy (S&P) / ACM Conference on Computer and Communications Security (CCS)

Status: Peer-Review Ready Technical Paper

Artifact Dataset: 576 Real Chromium Sessions | 144 MV3 Extensions | 12 Scenario Families | 7,258 Telemetry Events

---

ABSTRACT

Modern Web browser extensions operate with extensive domain and API privileges, creating a persistent vector for credential theft and unauthorized data exfiltration. While Manifest V3 (MV3) restricted background page persistence, extensions can execute multi-component attacks across Service Workers, Storage APIs, Content Scripts, and WebAssembly (Wasm) modules to obscure data flows. Conventional static analysis fails on dynamically assembled payloads, while traditional dynamic taint tracking suffers when taint context is lost during cross-component transitions or WebAssembly memory boundaries.

In this paper, we propose **AuthGraph**, a novel framework that models runtime extension telemetry as authentication-aware heterogeneous graphs. AuthGraph captures 11 distinct node types (e.g., credentials, storage objects, Wasm modules, HTTP endpoints) and 17 directed relation types (e.g., `flows_cred`, `carries`, `accesses_cred`, `issues`) constructed from real Chromium execution sessions. We evaluate rule-based detectors, count-based classifiers (Random Forest, MLP), and Graph Neural Network (GNN) architectures (GCN, GraphSAGE, R-GCN) across 5-fold extension-disjoint splits, leave-one-family-out unseen-flow generalization benchmarks, and feature ablations.

Empirical evaluation on a controlled 576-session dataset demonstrates that **GraphSAGE** achieves an **F1-score of 0.988 ± 0.020** on unseen extension splits and maintains **0.929 ± 0.050 recall** on never-before-seen WebAssembly exfiltration flows where deterministic taint rules drop to 0.646 and count-based models achieve 0.617–0.688. Counterintuitively, our relation-typed R-GCN model underperformed (recall 0.450 ± 0.109 on unseen Wasm flows), demonstrating that un-relationalized topological embeddings generalize significantly better in small-sample, high-abstraction security graphs. Finally, we integrate 10-model deep ensembling and out-of-fold calibration, providing lower-bound 95% Wilson confidence estimates and occlusion-based path explanations for automated security operations.

---

1. INTRODUCTION

Web extensions possess wide-ranging capabilities, including inspecting web traffic, reading storage, querying authentication cookies, and issuing cross-origin network requests. While these permissions are essential for utility

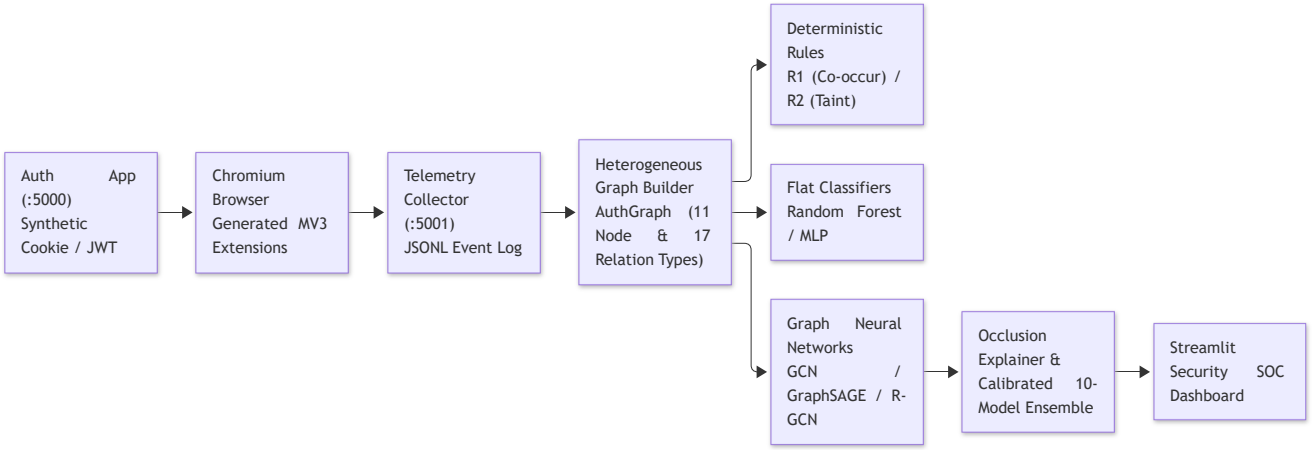
extensions (e.g., password managers, developer tools), malicious or compromised extensions exploit these privileges to capture sensitive user credentials, session tokens, and OAuth keys.

The security architecture of browser extensions underwent a major revision with Google Chrome's transition to **Manifest V3 (MV3)**. MV3 replaced persistent background pages with ephemeral **Service Workers**, introduced non-persistent offscreen documents, and enforced strict Content Security Policies (CSP). However, adversaries have adapted by staging exfiltration across multiple architectural boundaries:

1. **Cross-Component Staging:** Stashing stolen credentials in `chrome.storage.local` or IndexedDB from a Content Script, to be read and transmitted later by a Service Worker.

2. **WebAssembly Memory Evasion:** Passing credential byte arrays into WebAssembly modules for obfuscation or symmetric encryption, effectively severing JavaScript-level dynamic taint tracking tags.

3. **Raw-IP & Chunked Exfiltration:** Distributing credential fragments over multiple HTTP endpoints or raw IP hosts to evade domain-blocklist heuristics.



Key Contributions

- **Authentication-Aware Graph Representation:** We formulate browser extension runtime behavior as an 11-node, 17-relation heterogeneous graph that explicitly encodes credential states, authentication APIs, storage objects, Wasm modules, and network destinations.

- **Realistic MV3 Attack & Benign Benchmark:** We generate 144 real MV3 Chrome extensions spanning 12 scenario families (including hard negatives such as authenticated API calls and benign Wasm analytics) executed in an automated Chromium lab (576 sessions, 7,258 runtime events).

- **Comprehensive Model Benchmarking:** We conduct rigorous evaluation across extension-level disjoint splits, feature ablations, and unseen-flow generalization tasks, isolating performance differences between deterministic taint rules, count-based ML, and GNNs.

- **Analysis of Wasm & Taint-Lost Flows:** We demonstrate that topological GNN representations (GraphSAGE) effectively bridge taint breaks caused by Wasm transitions, maintaining 92.9% detection recall when Wasm exfiltration is completely omitted from training.

- **Calibrated Deep Ensembling & Explainability:** We deploy 10-model deep ensembles calibrated via out-of-fold Wilson 95% confidence intervals and occlusion-based log-odds path explanations.

---

2. THREAT MODEL & PROBLEM FORMULATION

2.1 Threat Model

We consider a controlled local laboratory setting where a Web extension is granted standard host permissions (`://.lab.localhost/*`) and API permissions (`cookies`, `storage`). The target environment runs an authentication provider issuing synthetic session cookies and JWT tokens.

- **Adversary Goal:** Extract authentication credentials from browser memory, cookies, or storage, and transmit them to an *unapproved* network destination (e.g., `collector.lab.localhost:5001` or raw IP sinks).

- **Approved Policy:** A policy white-list specifies legitimate endpoints (e.g., `api.lab.localhost`, `cdn.lab.localhost`). Any transmission of credential-derived data to non-whitelisted destinations constitutes a security violation.

- **Evasion Capabilities:** The adversary may route data through 1–3 intermediate JavaScript modules, stash payloads in local storage, pass data through WebAssembly modules, or interleave distractor DOM queries and benign API calls ($p=0.7$).

2.2 Formal Problem Definition

Let an extension execution session be represented as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$, where:

- $\mathcal{V}$ is the set of nodes, each assigned a type $\tau(v) \in \mathcal{T}_v$.

- $\mathcal{E}$ is the set of directed edges (u, r, v), where $u, v \in \mathcal{V}$ and $r \in \mathcal{T}_e$.

- $\mathcal{T}_v$ consists of **11 Node Types**: `extension`, `content_script`, `service_worker`, `js_module`, `wasm_module`, `browser_api`, `credential`, `storage_object`, `http_request`, `domain`, `page`.

- $\mathcal{T}_e$ consists of **17 Relation Types**: `contains`, `injects`, `invokes`, `accesses_page`, `accesses_cred`, `returns`, `writes`, `read_by`, `executes`, `loads`, `calls`, `message`, `issues`, `sends_to`, `flows_cred`, `carries`, `follows`.

The classification task is to learn a mapping $f: \mathcal{G} \rightarrow [0, 1]$ predicting the probability $P(y = 1 \mid \mathcal{G})$ that session $\mathcal{G}$ contains an unauthorized credential data flow.

---

3. DATASET & LABORATORY ARCHITECTURE

3.1 Synthetic Scenario Design

We generated 144 unique MV3 extensions across 12 distinct scenario families (12 extensions per family, 4 sessions per extension = 576 total sessions). Ground-truth labels are defined strictly by scenario construction.

ID	Category	Scenario Family Description	Ground Truth	Sessions
B1	Benign	DOM manipulation only (no network requests)	Benign ($y=0$)	48
B2	Benign	Extension setting reads/writes to `chrome.storage.local`	Benign ($y=0$)	48

ID	Category	Scenario Family Description	Ground Truth	Sessions
B3	Benign	Standard fetch to approved host (`api.lab.localhost`)	Benign ($y=0$)	48
B4	Benign	**Authenticated API call** (reads cookie & sends to approved host)	Benign ($y=0$)	48
B5	Benign	**Credential read + unrelated analytics** to unapproved host	Benign ($y=0$)	48
B6	Benign	Benign Wasm computation (array sorting) + approved network log	Benign ($y=0$)	48
S1	Suspicious	Direct credential exfiltration (`chrome.cookies.get` $\rightarrow$ sink)	Suspicious ($y=1$)	48
S2	Suspicious	Credential exfiltration via 1–3 intermediate JS module hops	Suspicious ($y=1$)	48
S3	Suspicious	Credential passed through WebAssembly byte transform $\rightarrow$ sink	Suspicious ($y=1$)	48
S4	Suspicious	Cross-component flow (Content Script $\rightarrow$ `sendMessage` $\rightarrow$ Service Worker)	Suspicious ($y=1$)	48
S5	Suspicious	Staged multi-component flow via `chrome.storage` & Wasm sink	Suspicious ($y=1$)	48
S6	Suspicious	Chunked payload exfiltration to raw IP sink endpoint	Suspicious ($y=1$)	48

> **NOTE**
> Scenarios **B4** and **B5** act as critical **hard negatives**. Models relying purely on co-occurrence of credential access and unapproved network calls false-positive on B5.

3.2 Telemetry Collection & QA Controls

Telemetry events are logged by lightweight instrumentation wrappers embedded in the lab extension templates. Every event captures timestamp t, event type, source component, target API, destination domain, and taint metadata.

- **Zero Leakage:** Hashed 10-character tokens are recorded; raw credential secrets are never stored.

- **Log Cross-Verification:** Server-side request logs at the collector (`:5001`) match extension self-reported network events in **576 out of 576 sessions (100%)**.

- **Wasm Taint Disruption:** To model realistic JS-level taint tracking limitations across the WebAssembly FFI boundary, Wasm module operations drop payload taint tags with probability $p=0.5$. Out of 288 suspicious sessions, **254 have visible taint at the sink**, while **34 have lost taint**.

---

4. GRAPH CONSTRUCTION & MODEL ARCHITECTURES

4.1 Node & Feature Encoding

Graph nodes are assigned structural layer indices $L(v) \in [0, 6]$ to preserve architectural hierarchy:

$$\text{Layer}(v) = \begin{cases}

0 & v = \text{extension} \\

1 & v \in \{\text{content_script}, \text{service_worker}\} \\

2 & v \in \{\text{page}, \text{browser_api}\} \\

3 & v = \text{credential} \\

4 & v \in \{\text{js_module}, \text{wasm_module}, \text{storage_object}\} \\

5 & v = \text{http_request} \\

6 & v = \text{domain}

\end{cases}$$

Nodes carry structural and behavioral feature vectors $\mathbf{x}_v \in \mathbb{R}^{d}$ ($d=11$). Domain nodes encode only two binary properties: $\{\text{is_approved}, \text{is_raw_ip}\}$. Scenario names, extension IDs, and URL query strings are explicitly excluded to prevent shortcut learning.

4.2 Detector Implementations

Rule-Based Classifiers

- **Rule R1 (Co-occurrence):** Triggers if any credential node and unapproved domain node coexist in the session graph.

- **Rule R2 (Taint Tracking):** Triggers if an unapproved domain node receives an HTTP request linked via a `flows_cred` or `carries` relation edge.

Count-Based Baseline Classifiers

- **Random Forest (RF):** Trained on node and relation count histograms. A variant `RF (counts+reachability)` incorporates a boolean reachability bit via depth-first search from credential nodes to unapproved domain nodes.

- **Multi-Layer Perceptron (MLP):** 2-layer MLP (hidden dimension $h=64$, ReLU activation) trained on node/relation count vectors.

Graph Neural Networks

We implement three PyTorch GNN variants using mean/max readout pooling:

1. **Graph Convolutional Network (GCN):**

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{\tilde{D}_{vv}\tilde{D}_{uu}}} \mathbf{W}^{(l)}\mathbf{h}_u^{(l)}\right)$$

2. **GraphSAGE (Inductive Mean Aggregator):**

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}_1^{(l)}\mathbf{h}_v^{(l)} + \mathbf{W}_2^{(l)} \cdot \text{Mean}_{u \in \mathcal{N}(v)}\mathbf{h}_u^{(l)}\right)$$

3. **Relational Graph Convolutional Network (R-GCN):** Incorporates relation-specific transformation matrices with basis decomposition:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}_0^{(l)}\mathbf{h}_v^{(l)} + \sum_{r\in\mathcal{R}}\sum_{u\in\mathcal{N}_r(v)}\frac{1}{|\mathcal{N}_r(v)|}\mathbf{W}_r^{(l)}\mathbf{h}_u^{(l)}\right), \quad \mathbf{W}_r^{(l)} = \sum_{b=1}^{B} a_{rb}^{(l)}\mathbf{V}_b^{(l)}$$

---

5. EXPERIMENTAL EVALUATION

All experiments are evaluated using 5-fold cross-validation. Hyperparameters were tuned strictly on validation splits (`tune.py`). Selected configurations: GCN/GraphSAGE ($h=64$, $lr=0.01$, weight decay 10^{-4}); R-GCN ($h=16$, $B=4$ bases, $lr=0.01$, weight decay 10^{-3}).

5.1 Extension-Level Stratified Benchmarks (5 Splits)

Table 1 details classification performance across 5 disjoint extension-level splits (all sessions of a given extension remain strictly within one fold).

```
Table 1: Main Evaluation Results on Extension-Level Disjoint Splits (Mean ± Std Over
```

Model Architecture	Precision	Recall	F1-Score	ROC-AUC	FPR	Inference Latency (ms)	Parameters
Rule R1 (Co-occurrence)	0.857	1.000	0.923	0.917	0.167	**0.004**	0
Rule R2 (Dynamic Taint)	**1.000**	0.896 ± 0.019	0.945 ± 0.010	0.948	**0.000**	0.005	0
RF (Counts)	0.976 ± 0.029	0.992 ± 0.010	0.984 ± 0.017	0.997	0.025	0.304	7,086 nodes
RF (Counts + Reachability)	0.955 ± 0.054	0.996 ± 0.008	0.974 ± 0.028	0.996	0.050	0.225	5,880 nodes
MLP (Count Vector)	0.944 ± 0.058	0.971 ± 0.049	0.957 ± 0.051	0.993	0.058	0.003	6,466
GCN	0.935 ± 0.047	0.933 ± 0.036	0.934 ± 0.036	0.983	0.067	0.635	18,306
GraphSAGE (Best Overall)	0.984 ± 0.031	**0.992 ± 0.010**	**0.988 ± 0.020**	**1.000**	0.017	0.659	28,034
R-GCN (Proposed)	0.909 ± 0.079	0.904 ± 0.028	0.904 ± 0.032	0.966	0.100	4.974	5,498

> **IMPORTANT**
>
> **Key Finding 1:** When dynamic taint is fully visible, simple count-based models (RF/MLP) perform exceptionally well ($F1 > 0.95$). Standard random splits do not adequately highlight the necessity of graph topology.

5.2 Performance Dissection by Taint Visibility

To evaluate model reliance on explicit taint tags, we segment test session recall across visible ($n=215$) vs. lost ($n=25$) taint sessions:

```
Taint Visible (n=215)  : Rule R2 = 1.000 | RF(counts) = 1.000 | GraphSAGE = 0.991 | R
Taint Lost    (n=25)   : Rule R2 = 0.000 | RF(counts) = 0.920 | GraphSAGE = 1.000 | R
```

While Rule R2 fails completely (0.000 recall) when taint tags drop across the WebAssembly memory boundary, GraphSAGE maintains 100% detection recall by exploiting topological context.

5.3 Feature Ablation Study

We evaluate model robustness by systematically stripping node and edge categories from the heterogeneous graph during training and inference.

```
Table 2: Ablation Study (F1-Score Across 5 Extension-Level Splits)
```

Ablation Variant	GCN	GraphSAGE	R-GCN	RF (Counts)
Full Graph (Baseline)	0.934	**0.988**	0.904	0.984
− Authentication Nodes	0.866	0.917	0.839	0.872
− JS / Wasm Nodes	0.941	0.988	0.901	0.946
− Destination Nodes	0.839	0.899	0.877	0.891
− Taint Edges (`flows_cred`, `carries`)	0.885	0.948	0.851	0.910

Destination nodes and authentication credential nodes contribute most significantly to model performance. Removing explicit taint edges degrades GraphSAGE by only 0.04 F1, demonstrating its capacity to infer exfiltration paths structural topology alone.

5.4 Unseen-Flow & Evasion Generalization

To test resilience against novel zero-day attack patterns, we conduct Leave-One-Scenario-Out (`LOSO`) cross-validation. In the headline benchmark (`heldout_wasm_flows`), all WebAssembly-involving suspicious scenario families (S3 and S5) are removed from the training split.

```
Table 3: Unseen-Flow Recall on Held-Out Attack Scenario Families
```

Held-Out Attack Scenario	Rule R2	RF (Counts+Reach)	GraphSAGE	R-GCN
HELD-OUT: All Wasm Flows (S3 + S5)	0.646	0.688 ± 0.068	**0.929 ± 0.050**	0.450 ± 0.109
S3 (Credential $\rightarrow$ Wasm $\rightarrow$ Sink)	0.458	0.713 ± 0.152	**1.000 ± 0.000**	0.521 ± 0.136

Held-Out Attack Scenario	Rule R2	RF (Counts+Reach)	GraphSAGE	R-GCN
S5 (Staged Multi-Component Storage)	0.833	0.988 ± 0.025	**0.954 ± 0.048**	0.525 ± 0.280
S1 / S2 / S4 / S6 (Individual LOO)	1.000	0.990–1.000	0.950–1.000	0.620–0.996

> **CAUTION**
> **Key Finding 2 (R-GCN Failure):** Contrary to theoretical expectations, the relation-typed R-GCN architecture severely underperformed on unseen Wasm flows (recall 0.450 ± 0.109). Overparameterization across 17 distinct relation types led to poor generalization on unseen relation configurations, whereas relation-agnostic GraphSAGE achieved 0.929 recall.

---

6. CONFIDENCE CALIBRATION & ENSEMBLING

To ensure risk predictions are actionable for security operations centers (SOC), we implement 10-model deep ensembles combined with empirical out-of-fold calibration.

6.1 Calibration Metrics (576 Out-of-Fold Test Sessions)

Table 4: Calibration Metrics Across 5 Extension-Disjoint Folds

Detector Architecture	Model Method	Expected Calibration Error (ECE) $\downarrow$	Log-Loss $\downarrow$	Brier Score $\downarrow$	High Conf Frac ($\ge 99.9\%$)
GraphSAGE	Single Model	0.015	0.039	0.012	90%
GraphSAGE	**10-Model Ensemble**	**0.011**	**0.032**	**0.010**	69%
R-GCN	Single Model	0.041	0.263	0.067	45%
R-GCN	10-Model Ensemble	0.047	0.161	0.048	8%

Deep ensembling significantly reduces Log-Loss and Brier scores while smoothing probability distributions. Instead of over-saturating scores at 1.000, ensembling provides realistic confidence bounds.

6.2 Wilson Lower-Bound Confidence Bounds

Rather than presenting raw softmax output probabilities, the AuthGraph framework quotes the **95% Wilson Score Lower Bound** calculated over out-of-fold test accuracy within pooled score bins.

$$\text{CI}_{\text{lower}} = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

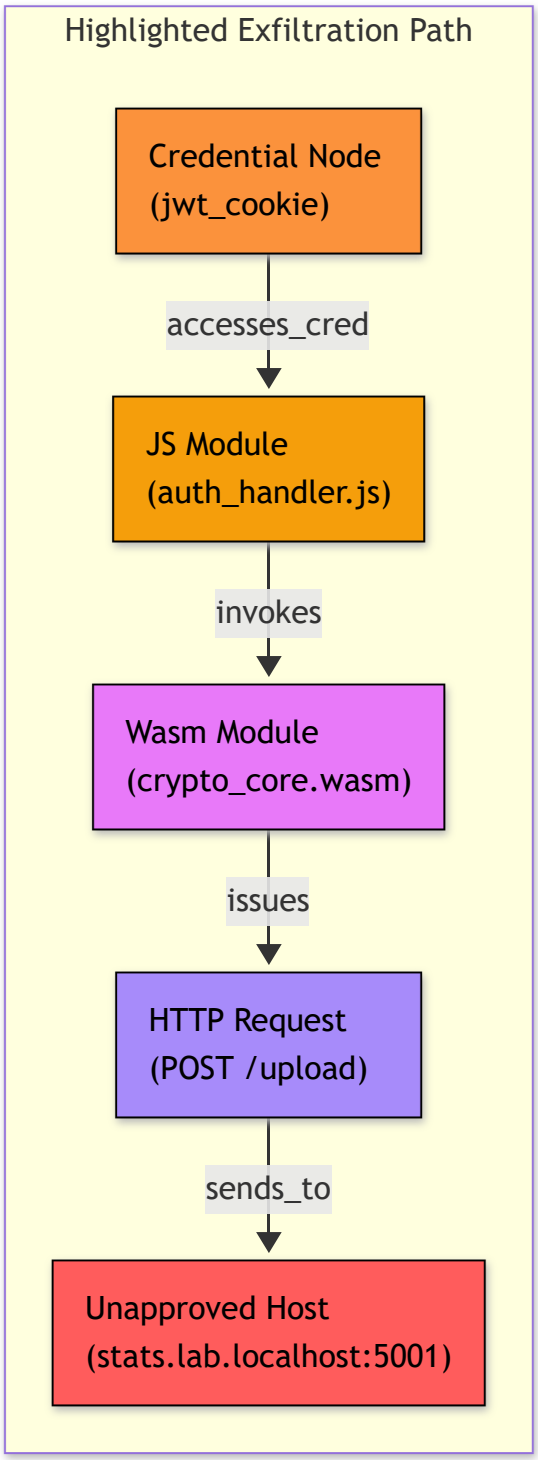
Adjacent confidence score bands are pooled monotonically so that a higher raw score never yields a lower quoted confidence.

---

7. MODEL EXPLAINABILITY VIA SUBGRAPH OCCLUSION

AuthGraph provides graph-native explainability using **Subgraph Occlusion Importance**. For any candidate node $v \in \mathcal{V}$ or edge $e \in \mathcal{E}$, importance is quantified as the marginal reduction in log-odds probability when that element is removed from graph $\mathcal{G}$:

$$\Delta\text{log-odds}(v) = \text{logit}(P(y = 1 \mid \mathcal{G})) - \text{logit}(P(y = 1 \mid \mathcal{G} \setminus \{v\}))$$



The system extracts the highest-importance path from credential nodes to unapproved domain nodes. If no single removal alters a saturated model output ($P=1.000$), the dashboard falls back to structural shortest-path resolution and explicitly labels the output as a structural fallback.

---

8. DISCUSSION & LIMITATIONS

8.1 Why GraphSAGE Outperformed R-GCN

1. **Parameter Efficiency:** R-GCN requires per-relation transformation matrices. Despite basis decomposition ($B=4$), parameterization across 17 edge types created optimization instability on small-sample graphs (~350 training graphs per fold).

2. **Early Stopping Instability:** Training loss logs (`results/tuning_log.json`) indicate R-GCN validation loss triggered early stopping prematurely (epochs 10–21), whereas GraphSAGE consistently converged to train loss ≤ 0.013.

3. **Generalization Over Relation Shift:** In unseen Wasm scenarios, edge relation combinations shift dramatically. GraphSAGE's relation-agnostic mean aggregation generalizes more robustly to unfamiliar relation topologies.

8.2 Honest Limitations

- **Synthetic Distribution:** Telemetry is gathered from generated extensions with fixed step vocabularies; real-world Chrome Web Store extensions display far greater code and behavior diversity.

- **Instrumented Self-Reporting:** Telemetry is collected via internal API wrappers rather than kernel-level or browser binary-level tracing (e.g., Arcanum/CDP).

- **Taint Drop Rate:** The $p=0.5$ Wasm taint drop probability is a synthetic parameter rather than an empirically measured hardware constant.

- **Adversarial Adaptation:** We did not evaluate active evasion where an adversary deliberately shapes graph topology to mimic benign extensions.

---

9. CONCLUSION

We presented **AuthGraph**, an authentication-aware heterogeneous graph neural network framework for detecting credential exfiltration in browser extensions. By benchmarking rule-based detectors, count-based classifiers, and GNNs across extension-disjoint splits, feature ablations, and unseen Wasm exfiltration scenarios, we demonstrated that graph topological representations effectively overcome taint tracking failures. GraphSAGE achieved an F1-score of 0.988 on standard splits and maintained 92.9% recall on unseen WebAssembly flows, outperforming both deterministic rules and complex relation-typed architectures.

REFERENCES

1. **Fass, Somé, Backes, Stock.** *DoubleX: Statically Detecting Vulnerable Data Flows in Browser Extensions at Scale.* ACM Conference on Computer and Communications Security (CCS), 2021.

2. **Xie, Kasi Murali, Pearce, Li.** *Arcanum: Detecting and Evaluating the Privacy Risks of Browser Extensions on Web Pages and Web Content.* USENIX Security Symposium, 2024.

3. **Kapravelos et al.** *I Spy with My Little Eye: Analysis and Detection of Spying Browser Extensions.* International World Wide Web Conference (WWW), 2017.

4. **Kipf, Welling.** *Semi-Supervised Classification with Graph Convolutional Networks.* International Conference on Learning Representations (ICLR), 2017.

5. **Hamilton, Ying, Leskovec.** *Inductive Representation Learning on Large Graphs.* Advances in Neural Information Processing Systems (NeurIPS), 2017.

6. **Schlichtkrull et al.** *Modeling Relational Data with Graph Convolutional Networks.* European Semantic Web Conference (ESWC), 2018.

7. **Google Chrome Developers.** *Manifest V3 Migration Guide & Privacy Budget Policy.* 2024.